

Universidad Mariano Gálvez
Facultad de ingeniería en sistemas
Programación



Brayan Daniel Barrios Carranza
Carné: 9941-15-6843

Introducción

En este proyecto se desarrolló un sistema de gestión de ventas e inventario para una tienda utilizando el lenguaje de programación C++. El objetivo principal fue aplicar los conocimientos aprendidos durante el curso, especialmente el uso de estructuras, archivos binarios, búsquedas, ordenamientos y manejo de excepciones.

El sistema permite registrar productos, realizar ventas, actualizar el inventario automáticamente y generar distintos reportes que facilitan el control de la información. Además, los datos se almacenan en archivos binarios para que puedan conservarse aunque el programa se cierre.

Durante el desarrollo del proyecto se pusieron en práctica diferentes técnicas de programación estructurada y modularización, buscando que el sistema fuera funcional y fácil de utilizar.

Objetivo General

Desarrollar un sistema de gestión de ventas e inventario utilizando C++ y archivos binarios, aplicando los conocimientos adquiridos durante el curso de Programación II.

Objetivos Específicos

- Implementar el registro y administración de productos.
- Controlar el inventario mediante actualización automática de existencias.
- Registrar ventas con múltiples productos.
- Aplicar cálculos de IVA, descuentos y totales.
- Generar reportes estadísticos de productos y ventas.
- Utilizar archivos binarios para almacenar información permanentemente.
- Aplicar búsquedas, ordenamientos y validaciones de datos.

Estructuras Utilizadas

Estructura Producto

La estructura Producto almacena la información principal de cada artículo registrado dentro del inventario.

Campos utilizados:

- Código
- Nombre
- Precio
- Stock
- Cantidad vendida

Estructura Detalle de Venta

La estructura Detalle de venta almacena los productos que forman parte de una venta.

Campos utilizados:

- Código del producto
- Nombre
- Cantidad vendida
- Precio unitario
- Subtotal

Estructura Venta

La estructura Venta almacena la información general de cada transacción realizada.

Campos utilizados:

- Número de venta
- Día
- Mes
- Año
- Cantidad de artículos
- Subtotal
- IVA
- Descuento
- Total final

Archivos Utilizados

El sistema utiliza archivos binarios para almacenar información de forma permanente.

Archivo	Descripción
productos.dat	Almacena los productos registrados
ventas.dat	Almacena las ventas realizadas
detalleVentas.dat	Guarda el detalle de cada venta
reporte.txt	Reportes exportados en texto

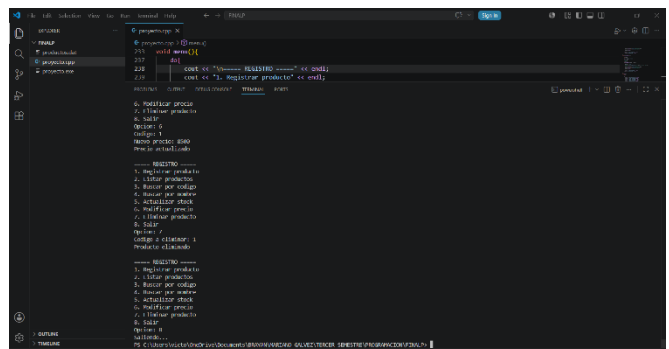
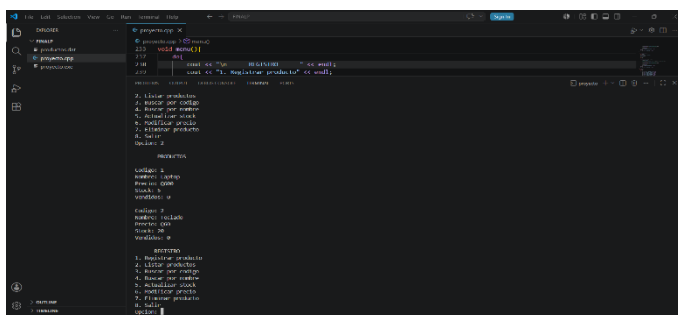
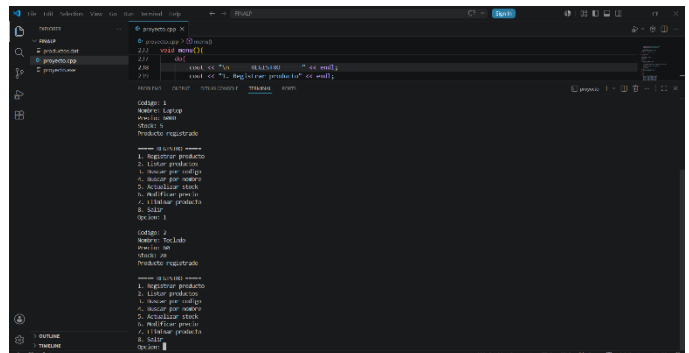
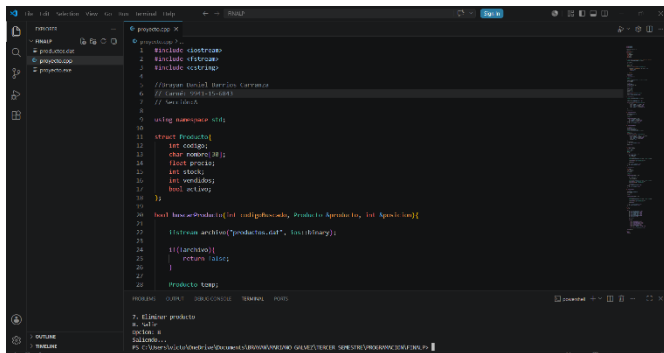
Módulo 1 – Gestión de Productos

Este módulo permite administrar los productos registrados dentro del inventario.

Funciones implementadas:

- Registrar productos
- Listar productos
- Buscar por código
- Buscar por nombre
- Actualizar stock
- Modificar precio
- Eliminar productos

Evidencias



Módulo 2 – Proceso de Ventas

Este módulo permite registrar ventas y actualizar automáticamente el inventario.

Funciones implementadas:

- Creación de ventas
- Validación de stock disponible
- Registro de múltiples productos
- Aplicación de IVA
- Aplicación de descuentos
- Actualización automática del stock
- Almacenamiento en archivos binarios

Fórmulas utilizadas

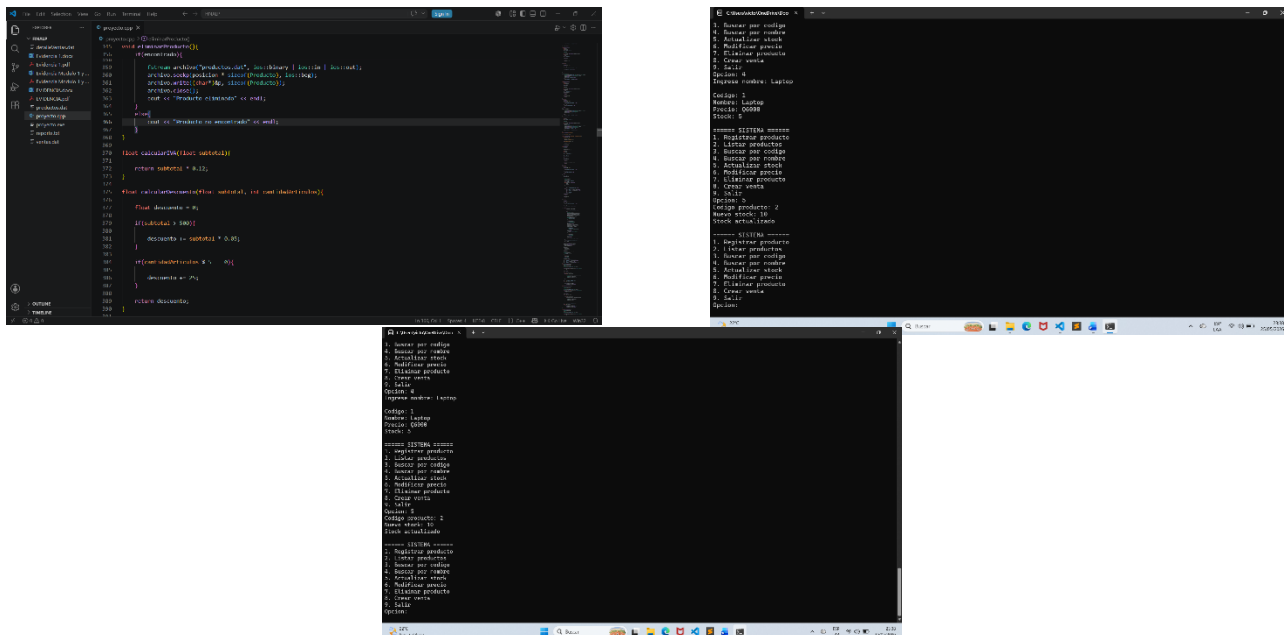
IVA:

$$IVA = Subtotal \times 0.12$$

Descuento por compra mayor a Q500:

$$Descuento = Subtotal \times 0.05$$

Evidencia



Módulo 3 – Reportes

Este módulo permite generar información estadística sobre el sistema.

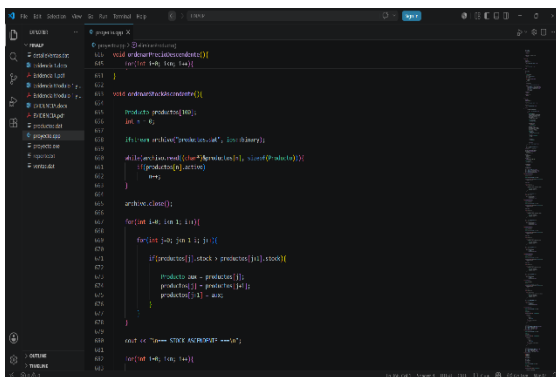
Reportes implementados:

- Productos con menor stock
- Productos más vendidos
- Ventas totales del día
- Ventas por mes mediante matriz
- Ordenamiento por precio ascendente
- Ordenamiento por precio descendente
- Ordenamiento por stock ascendente
- Ordenamiento por stock descendente
- Ordenamiento por ventas acumuladas

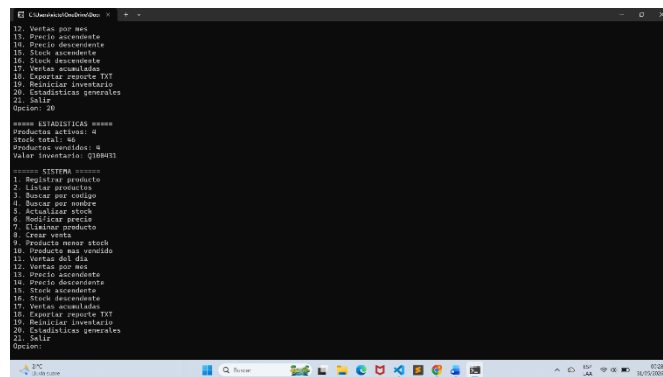
Algoritmos utilizados

Para los ordenamientos se implementó el algoritmo Bubble Sort.

Evidencia



```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 using namespace std;
6 struct Producto {
7     string nombre;
8     int stock;
9     float precio;
10 };
11 vector<Producto> productos;
12 void agregarProducto() {
13     Producto p;
14     cout << "Nombre: ";
15     getline(cin, p.nombre);
16     cout << "Stock: ";
17     int stock;
18     while (true) {
19         cin >> stock;
20         if (stock > 0) break;
21     }
22     cout << "Precio: ";
23     float precio;
24     while (true) {
25         cin >> precio;
26         if (precio > 0) break;
27     }
28     p.stock = stock;
29     p.precio = precio;
30     productos.push_back(p);
31 }
32 void listarProductos() {
33     if (productos.empty()) {
34         cout << "No hay productos registrados." << endl;
35     } else {
36         cout << "Lista de productos:" << endl;
37         for (int i = 0; i < productos.size(); i++) {
38             cout << "ID: " << i << ", Nombre: " << productos[i].nombre << ", Stock: " << productos[i].stock << ", Precio: " << productos[i].precio << endl;
39         }
40     }
41 }
42 void buscarProducto() {
43     string nombre;
44     cout << "Ingrese nombre del producto: ";
45     getline(cin, nombre);
46     bool encontrado = false;
47     for (int i = 0; i < productos.size(); i++) {
48         if (productos[i].nombre == nombre) {
49             encontrado = true;
50             cout << "Producto encontrado. Stock: " << productos[i].stock << endl;
51         }
52     }
53     if (!encontrado) {
54         cout << "Producto no encontrado." << endl;
55     }
56 }
57 void actualizarStock() {
58     string nombre;
59     int stock;
60     cout << "Ingrese nombre del producto: ";
61     getline(cin, nombre);
62     bool encontrado = false;
63     for (int i = 0; i < productos.size(); i++) {
64         if (productos[i].nombre == nombre) {
65             encontrado = true;
66             cout << "Ingrese nuevo stock: ";
67             int nStock;
68             while (true) {
69                 cin >> nStock;
70                 if (nStock > 0) break;
71             }
72             productos[i].stock = nStock;
73             cout << "Stock actualizado." << endl;
74         }
75     }
76     if (!encontrado) {
77         cout << "Producto no encontrado." << endl;
78     }
79 }
80 void eliminarProducto() {
81     string nombre;
82     cout << "Ingrese nombre del producto a eliminar: ";
83     getline(cin, nombre);
84     bool encontrado = false;
85     for (int i = 0; i < productos.size(); i++) {
86         if (productos[i].nombre == nombre) {
87             encontrado = true;
88             productos.erase(productos.begin() + i);
89             cout << "Producto eliminado." << endl;
90             break;
91         }
92     }
93     if (!encontrado) {
94         cout << "Producto no encontrado." << endl;
95     }
96 }
97 int main() {
98     int opcion;
99     do {
100         cout << "Menu de opciones:" << endl;
101         cout << "1. Registrar producto" << endl;
102         cout << "2. Listar productos" << endl;
103         cout << "3. Buscar por código" << endl;
104         cout << "4. Actualizar stock" << endl;
105         cout << "5. Eliminar producto" << endl;
106         cout << "6. Salir" << endl;
107         cout << "Opcion: ";
108         int opcion;
109         while (true) {
110             cin >> opcion;
111             if (opcion > 0 && opcion <= 6) break;
112         }
113         switch (opcion) {
114             case 1: agregarProducto(); break;
115             case 2: listarProductos(); break;
116             case 3: buscarProducto(); break;
117             case 4: actualizarStock(); break;
118             case 5: eliminarProducto(); break;
119             case 6: break;
120         }
121     } while (opcion != 6);
122     return 0;
123 }
```



```
12. Ventas por mes
13. Precio ascendente
14. Precio descendente
15. Stock ascendente
16. Stock descendente
17. Ventas acumuladas
18. Exportar reporte TXT
19. Reiniciar inventario
20. Estadísticas generales
21. Salir
Opcion: 20

===== ESTADÍSTICAS =====
Productos activos: 4
Stock total: 16
Productos vendidos: 4
Valor inventario: 108041

===== MENU =====
1. Registrar producto
2. Listar productos
3. Buscar por código
4. Actualizar stock
5. Eliminar producto
6. Salir
Opcion: 1
Producto nuevo stock
12. Ventas por mes
13. Precio ascendente
14. Precio descendente
15. Stock ascendente
16. Stock descendente
17. Ventas acumuladas
18. Exportar reporte TXT
19. Reiniciar inventario
20. Estadísticas generales
21. Salir
Opcion:
```

Módulo 4 – Utilidades del Sistema

Este módulo proporciona herramientas complementarias para la administración del sistema.

Funciones implementadas:

- Exportación de reportes a archivo TXT.
- Reinicio del inventario.
- Estadísticas generales del sistema.

Evidencia

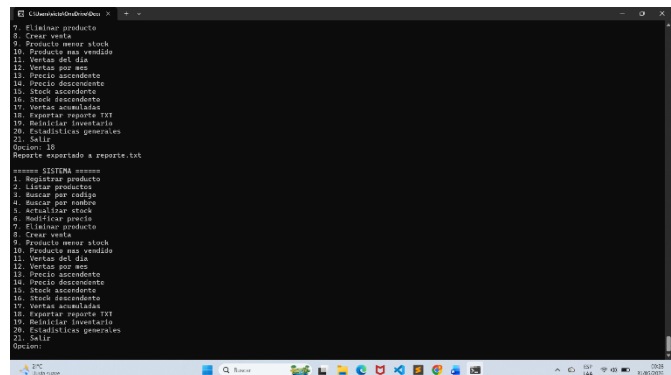
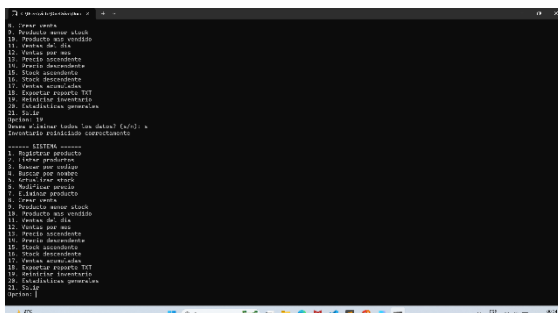
```
)
}
conf << „operation canceled“ << endl;
else{
}
conf << „Inventory reset complete“ << endl;

switch(cjose){
case 1:
break;
case 2:
break;
case 3:
break;
case 4:
break;
case 5:
break;
case 6:
break;
case 7:
break;
case 8:
break;
case 9:
break;
case 10:
break;
case 11:
break;
case 12:
break;
case 13:
break;
case 14:
break;
case 15:
break;
case 16:
break;
case 17:
break;
case 18:
break;
case 19:
break;
case 20:
break;
case 21:
break;
case 22:
break;
case 23:
break;
case 24:
break;
case 25:
break;
case 26:
break;
case 27:
break;
case 28:
break;
case 29:
break;
case 30:
break;
case 31:
break;
case 32:
break;
case 33:
break;
case 34:
break;
case 35:
break;
case 36:
break;
case 37:
break;
case 38:
break;
case 39:
break;
case 40:
break;
case 41:
break;
case 42:
break;
case 43:
break;
case 44:
break;
case 45:
break;
case 46:
break;
case 47:
break;
case 48:
break;
case 49:
break;
case 50:
break;
case 51:
break;
case 52:
break;
case 53:
break;
case 54:
break;
case 55:
break;
case 56:
break;
case 57:
break;
case 58:
break;
case 59:
break;
case 60:
break;
case 61:
break;
case 62:
break;
case 63:
break;
case 64:
break;
case 65:
break;
case 66:
break;
case 67:
break;
case 68:
break;
case 69:
break;
case 70:
break;
case 71:
break;
case 72:
break;
case 73:
break;
case 74:
break;
case 75:
break;
case 76:
break;
case 77:
break;
case 78:
break;
case 79:
break;
case 80:
break;
case 81:
break;
case 82:
break;
case 83:
break;
case 84:
break;
case 85:
break;
case 86:
break;
case 87:
break;
case 88:
break;
case 89:
break;
case 90:
break;
case 91:
break;
case 92:
break;
case 93:
break;
case 94:
break;
case 95:
break;
case 96:
break;
case 97:
break;
case 98:
break;
case 99:
break;
case 100:
break;
}

if(option == ,2, || option == ,2,){
cin >> option;
conf << „Press enter to continue“ << endl;
}

return option;
}

int main(){
    int option;
    while(option != 0){
        option = menu();
        switch(option){
            case 1:
                register_product();
            case 2:
                list_products();
            case 3:
                search_by_name();
            case 4:
                search_by_sku();
            case 5:
                modify_price();
            case 6:
                delete_product();
            case 7:
                create_sale();
            case 8:
                sale_less_stock();
            case 9:
                sale_more_stock();
            case 10:
                sale_by_date();
            case 11:
                sale_by_day();
            case 12:
                sale_by_month();
            case 13:
                sale_by_year();
            case 14:
                sale_by_quarter();
            case 15:
                sale_by_half_year();
            case 16:
                sale_by_full_year();
            case 17:
                sale_by_all_years();
            case 18:
                sale_by_all_months();
            case 19:
                sale_by_all_days();
            case 20:
                sale_by_all_hours();
            case 21:
                sale_by_all_minutes();
            case 22:
                sale_by_all_seconds();
            case 23:
                sale_by_all_milliseconds();
            case 24:
                sale_by_all_microseconds();
            case 25:
                sale_by_all_nanoseconds();
            case 26:
                sale_by_all_picoseconds();
            case 27:
                sale_by_all_femtoseconds();
            case 28:
                sale_by_all_attoseconds();
            case 29:
                sale_by_allzeptoseconds();
            case 30:
                sale_by_all_yoctoseconds();
            case 31:
                sale_by_all_rontoseconds();
            case 32:
                sale_by_all_hertzs();
            case 33:
                sale_by_all_kilohertzs();
            case 34:
                sale_by_all_megahertzs();
            case 35:
                sale_by_all_gigahertzs();
            case 36:
                sale_by_all_terahertzs();
            case 37:
                sale_by_all_petahertzs();
            case 38:
                sale_by_all_exahertzs();
            case 39:
                sale_by_all_zettahertzs();
            case 40:
                sale_by_all_yottahertzs();
            case 41:
                sale_by_all_ronnahertzs();
            case 42:
                sale_by_all_septahertzs();
            case 43:
                sale_by_all_octahertzs();
            case 44:
                sale_by_all_ninahertzs();
            case 45:
                sale_by_all_decahertzs();
            case 46:
                sale_by_all_unithertzs();
            case 47:
                sale_by_all_dekahertzs();
            case 48:
                sale_by_all_hectohertzs();
            case 49:
                sale_by_all_kiloherths();
            case 50:
                sale_by_all_megaherths();
            case 51:
                sale_by_all_gigaherths();
            case 52:
                sale_by_all_teraherths();
            case 53:
                sale_by_all_petaherths();
            case 54:
                sale_by_all_exaherths();
            case 55:
                sale_by_all_zettaherths();
            case 56:
                sale_by_all_yottaherths();
            case 57:
                sale_by_all_ronnaherths();
            case 58:
                sale_by_all_septaherths();
            case 59:
                sale_by_all_octaherths();
            case 60:
                sale_by_all_ninaherths();
            case 61:
                sale_by_all_decaherths();
            case 62:
                sale_by_all_unitherths();
            case 63:
                sale_by_all_dekaherths();
            case 64:
                sale_by_all_hectaherths();
            case 65:
                sale_by_all_kiloherths();
            case 66:
                sale_by_all_megaherths();
            case 67:
                sale_by_all_gigaherths();
            case 68:
                sale_by_all_teraherths();
            case 69:
                sale_by_all_petaherths();
            case 70:
                sale_by_all_exaherths();
            case 71:
                sale_by_all_zettaherths();
            case 72:
                sale_by_all_yottaherths();
            case 73:
                sale_by_all_ronnaherths();
            case 74:
                sale_by_all_septaherths();
            case 75:
                sale_by_all_octaherths();
            case 76:
                sale_by_all_ninaherths();
            case 77:
                sale_by_all_decaherths();
            case 78:
                sale_by_all_unitherths();
            case 79:
                sale_by_all_dekaherths();
            case 80:
                sale_by_all_hectaherths();
            case 81:
                sale_by_all_kiloherths();
            case 82:
                sale_by_all_megaherths();
            case 83:
                sale_by_all_gigaherths();
            case 84:
                sale_by_all_teraherths();
            case 85:
                sale_by_all_petaherths();
            case 86:
                sale_by_all_exaherths();
            case 87:
                sale_by_all_zettaherths();
            case 88:
                sale_by_all_yottaherths();
            case 89:
                sale_by_all_ronnaherths();
            case 90:
                sale_by_all_septaherths();
            case 91:
                sale_by_all_octaherths();
            case 92:
                sale_by_all_ninaherths();
            case 93:
                sale_by_all_decaherths();
            case 94:
                sale_by_all_unitherths();
            case 95:
                sale_by_all_dekaherths();
            case 96:
                sale_by_all_hectaherths();
            case 97:
                sale_by_all_kiloherths();
            case 98:
                sale_by_all_megaherths();
            case 99:
                sale_by_all_gigaherths();
            case 100:
                sale_by_all_teraherths();
        }
    }
}
```



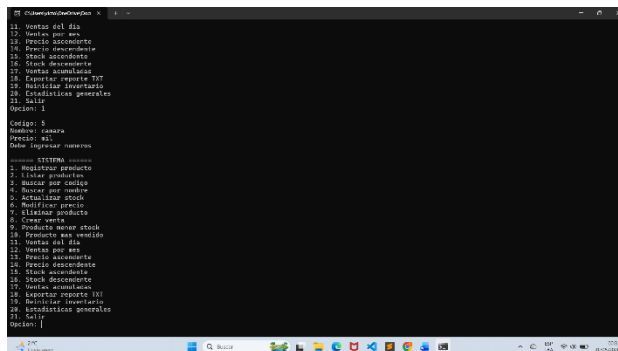
Validaciones y Manejo de Excepciones

Para garantizar el correcto funcionamiento del sistema se implementaron validaciones mediante estructuras try/catch.

Validaciones realizadas:

- Precio inválido.
- Stock negativo.
- Cantidades inválidas.
- Mes fuera de rango.
- Entradas no numéricas.

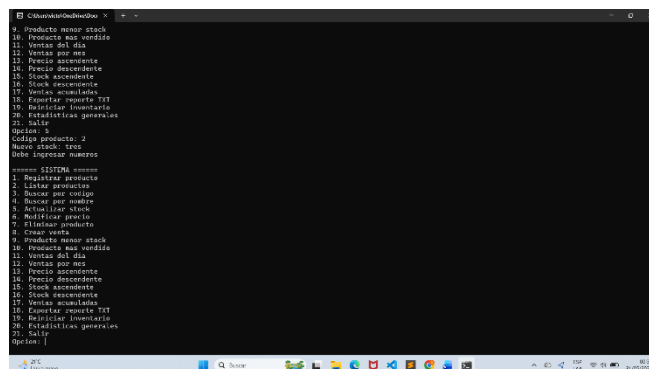
Estas validaciones permiten evitar errores durante la ejecución y mejorar la confiabilidad del sistema.



```
11. Ventas del día
12. Ventas por mes
13. Precio ascendente
14. Precio ascendente
15. Stock descendente
16. Stock descendente
17. Ventas acumuladas
18. Reporte TTT
19. Modificar inventario
20. Cuentas generales
21. Salir
Opcion: 1

Codigo: 5
Nombre: Zanahora
Precio: 100
Debe ingresar numeros

===== SISTEMA =====
1. Registrar producto
2. Listar productos
3. Buscar por código
4. Buscar por nombre
5. Actualizar stock
6. Modificar precio
7. Eliminar producto
8. Crear venta
9. Producto menor stock
10. Producto mas vendido
11. Ventas del día
12. Ventas por mes
13. Precio ascendente
14. Precio descendente
15. Stock descendente
16. Stock descendente
17. Ventas acumuladas
18. Reporte TTT
19. Modificar inventario
20. Estadísticas generales
21. Salir
Opcion: 1
```



```
9. Producto menor stock
10. Producto mas vendido
11. Ventas del día
12. Ventas por mes
13. Precio ascendente
14. Precio descendente
15. Stock descendente
16. Stock descendente
17. Ventas acumuladas
18. Reporte TTT
19. Modificar inventario
20. Estadísticas generales
21. Salir
Opcion: 1

Codigo producto: 1
Nuevo stock: 100
Debe ingresar numeros

===== SISTEMA =====
1. Registrar producto
2. Listar productos
3. Buscar por código
4. Buscar por nombre
5. Actualizar stock
6. Modificar precio
7. Eliminar producto
8. Crear venta
9. Producto menor stock
10. Producto mas vendido
11. Ventas del día
12. Ventas por mes
13. Precio ascendente
14. Precio descendente
15. Stock descendente
16. Stock descendente
17. Ventas acumuladas
18. Reporte TTT
19. Modificar inventario
20. Estadísticas generales
21. Salir
Opcion: 1
```

Conclusiones

1. Se desarrolló un sistema funcional de gestión de ventas e inventario utilizando programación estructurada y archivos binarios en C++.
2. El uso de estructuras, archivos binarios y funciones permitió organizar adecuadamente la información y mantener la persistencia de los datos.
3. La implementación de reportes, ordenamientos y validaciones contribuyó a mejorar el control y administración de los productos y ventas dentro del sistema.